Thuật toán Dijkstra

Tìm đường đi ngắn nhất trên đồ thị

Nhóm 7

Ngày 19 tháng 3 năm 2026

Nội dung

1. Bài toán
2. Ý tưởng thuật toán
3. Cấu trúc dữ liệu
4. Code thuật toán
5. Độ phức tạp
6. So sánh thuật toán
7. Kết luận

Bài toán đặt ra

Input

Đồ thị có trọng số không âm

Yêu cầu

Tìm khoảng cách ngắn nhất từ đỉnh nguồn s đến mọi đỉnh còn lại

Ứng dụng

- Bản đồ & GPS
- Mạng máy tính
- Tối ưu chi phí vận chuyển

Ý tưởng tham lam

- Luôn chọn đỉnh chưa xét có khoảng cách nhỏ nhất
- Cố định khoảng cách của đỉnh đó
- Cập nhật (relax) các đỉnh kề
- Lặp lại cho đến khi duyệt hết

Tư tưởng chính

Mở rộng dần vùng tối ưu từ đỉnh nguồn

Các bước chính

1. Khởi tạo:
$$dist[s] = 0, \quad dist[v] = \infty$$
2. Đưa $(0, s)$ vào priority queue
3. Lặp:
   - Lấy đỉnh có dist nhỏ nhất
   - Relax các cạnh kề
4. Kết thúc khi hàng đợi rỗng

Cấu trúc cần dùng

- Danh sách kề lưu đồ thị
- Mảng *dist*[] lưu khoảng cách ngắn nhất
- Priority queue chọn đỉnh gần nhất
- Mảng *visited*[] (tuỳ chọn)

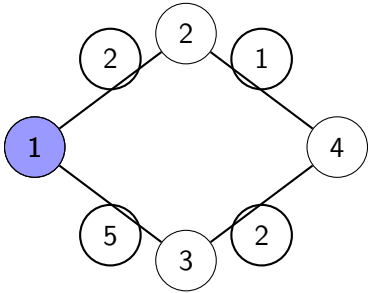
Tối ưu quan trọng

Dùng heap để giảm độ phức tạp

Code Dijkstra

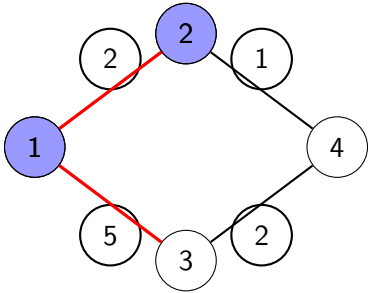
```
priority_queue<pll, vector<pll>, greater<pll>> pq;
ll dist(N, 1e18);
void dijkstra(ll start)
{
    dist[start] = 0;
    pq.push({0, start});
    while (!pq.empty())
   {
        auto [d, u] = pq.top();
        pq.pop();
        if (d > dist[u])
            continue;
        for (auto [v, w] : adj[u])
       {
            if (dist[u] + w < dist[v])
           {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
           }
       }
   }
}
```

Mô phỏng Dijkstra



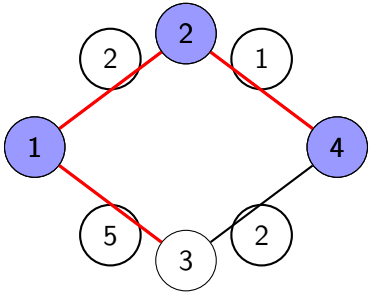
- Bước 1: Chọn đỉnh nguồn 1, $dist[1] = 0$

Mô phỏng Dijkstra



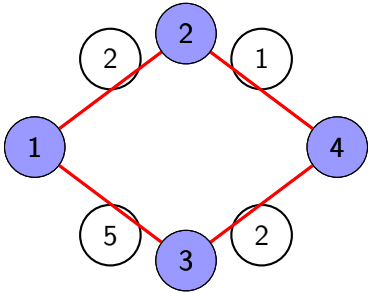
- Bước 1: Chọn đỉnh nguồn 1, $dist[1] = 0$
- Bước 2: Relax cạnh $(1 \to 2), (1 \to 3)$ $\quad dist[2] = 2,\ dist[3] = 5$

Mô phỏng Dijkstra



- Bước 1: Chọn đỉnh nguồn 1, $dist[1] = 0$
- Bước 2: Relax cạnh $(1 \to 2), (1 \to 3)$ $\quad dist[2] = 2,\ dist[3] = 5$
- Bước 3: Chọn đỉnh 2 (nhỏ nhất), relax $(2 \to 4)$ $\quad dist[4] = 3$

Mô phỏng Dijkstra



- Bước 1: Chọn đỉnh nguồn 1, $dist[1] = 0$
- Bước 2: Relax cạnh $(1 \to 2), (1 \to 3)$ $\quad dist[2] = 2,\ dist[3] = 5$
- Bước 3: Chọn đỉnh 2 (nhỏ nhất), relax $(2 \to 4)$ $\quad dist[4] = 3$
- Bước 4: Chọn đỉnh 4, relax $(4 \to 3)$ $\quad$ Không cải thiện

Phân tích độ phức tạp

Dùng mảng thường

$O(V^2)$

Dùng Priority Queue (Heap)

$O((V + E) \log V)$

Thực tế

Rất nhanh với đồ thị thưa

So sánh với thuật toán khác

Thuật toán	Trọng số âm	Độ phức tạp
BFS	Không	$O(V + E)$
Dijkstra	Không	$O((V + E) \log V)$
Bellman–Ford	Có	$O(VE)$
Floyd–Warshall	Có	$O(V^3)$

Tổng kết

- Dijkstra tìm đường đi ngắn nhất hiệu quả
- Áp dụng cho đồ thị trọng số không âm
- Cốt lõi: tham lam + heap
- Nền tảng cho nhiều bài toán tối ưu

Cảm ơn các bạn đã lắng nghe!